

Méthode des Éléments de Frontière (BEM) appliquée à l'équation de Helmholtz 2D

Clotilde Auber–Joignant, Antoine Levrat

1 Introduction

Ce travail s'inscrit dans la continuité des précédents compte-rendus, dans lesquels nous avons introduit le problème de diffraction d'une onde par un obstacle et une façon de résoudre ce type de problèmes via la méthode des éléments de frontière.

La résolution de l'équation de Helmholtz par la méthode des éléments de frontière (BEM) conduit à la résolution d'un système linéaire du type :

$$Ax = b,$$

où A est une matrice dense. Contrairement à la méthode des éléments finis (FEM), où la matrice obtenue est creuse, ce qui permet l'utilisation de solveurs rapides et efficaces, la matrice issue de la BEM est dense. Cette densité entraîne des coûts mémoire et temps de calcul en $O(N^2)$ pour le stockage et en $O(N^3)$ pour une factorisation directe.

Afin de rendre la résolution de tels systèmes accessible pour des tailles de problèmes réalistes, il est nécessaire d'introduire des approximations de la matrice A qui réduisent sa complexité. L'objectif de ce travail est d'explorer différentes méthodes rapides de factorisation de matrices, en particulier dans le cas où A est une matrice de rang faible (*i.e* le rang r de $A \in \mathbb{R}^{m \times n}$ est petit devant m et n .)

Note : Pour laisser plus de place à l'analyse de l'efficacité des différentes méthodes étudiées, dans ce papier nous réduirons les définitions de ces dernières au minimum.

2 Différentes méthodes de factorisation

2.1 Singular Value Decomposition (SVD)

La méthode la plus naturelle et robuste pour obtenir une approximation de rang faible d'une matrice est la *Singular Value Decomposition*. Elle consiste à factoriser une matrice $A \in \mathbb{R}^{m \times n}$ sous la forme :

$$A_k = U_k \Sigma_k V_k^T,$$

où $U_k \in \mathbb{R}^{m \times k}$ et $V_k^T \in \mathbb{R}^{k \times n}$ sont des matrices orthogonales, et $\Sigma_k \in \mathbb{R}^{k \times k}$ est une matrice diagonale

En termes de coût, le stockage passe de $O(mn)$ coefficients pour A à seulement $O(k(m+n))$ pour l'approximation A_k , ce qui représente un gain considérable lorsque $k \ll \min(m, n)$. Nous avons appliqué cette méthode, les résultats sont donnés en Figure 1.

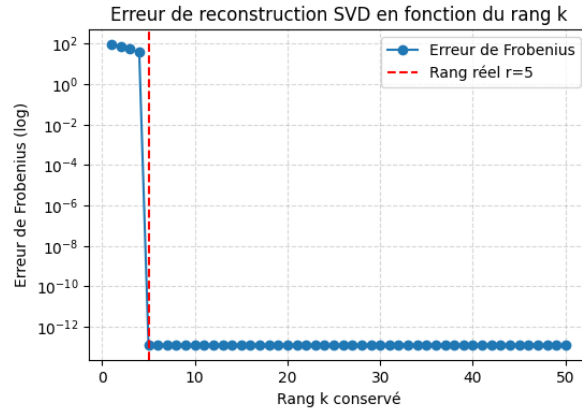


Figure 1 – Erreur en norme de Frobenius entre $A \in \mathbb{R}^{50 \times 50}$ de rang faible $r_f = 5$ et A_k pour $k \in [1, 2, \dots, 50]$

On observe que l'erreur entre A et A_k est grande tant que le rang gardé est plus petit que 5, mais en considérant un rang égal à 5 l'erreur décroît substantiellement, passant d'environ 10^2 à moins de 10^{-12} . L'erreur ne décroît plus pour $k > 5$, il est donc inutile de considérer un rang plus élevé que 5 pour approximer la matrice A .

Ainsi, la SVD tronquée semble apporter une solution satisfaisante à notre problème. Cependant, notre objectif est de réduire l'espace de stockage, mais aussi le coût de factorisation. Sur ce dernier point, nous avons besoin de calculer entièrement A pour déterminer la SVD. Le coût de factorisation reste donc très élevé, d'où la recherche d'une méthode moins coûteuse.

2.2 Adaptive Cross Approximation, cas complet (ACA_c)

La méthode ACA vise à obtenir une approximation d'une matrice sans avoir à la construire ni à la factoriser entièrement. L'idée est de sélectionner de manière itérative des lignes et colonnes pertinentes de la matrice A afin de construire une approximation sous la forme :

$$A_k = U_k V_k^T,$$

où $U_k \in \mathbb{R}^{m \times k}$ et $V_k \in \mathbb{R}^{n \times k}$ sont formés à partir des colonnes et lignes extraites de A .

Dans le cas complet (ACA_c), le pivot choisi à chaque itération correspond à l'élément de plus grande valeur absolue du résidu :

$$R_{ij}^{(t)} = A_{ij} - (U_t V_t^T)_{ij}.$$

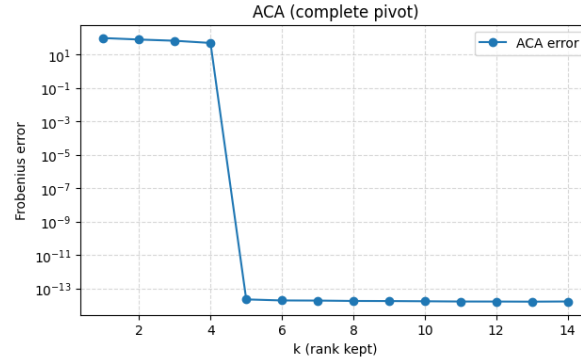


Figure 2 – Erreur en norme de Frobenius entre $A \in \mathbb{R}^{50 \times 50}$ de rang faible $r_f = 5$ pour $k \in [1, 2, \dots, 14]$ et son approximation A_k obtenue par ACA_c .

On observe une décroissance similaire à celle obtenue avec la SVD tronquée : l'erreur devient quasi nulle dès que le rang d'approximation dépasse le rang réel $r_f = 5$.

L' ACA_c permet d'obtenir une approximation de qualité comparable à la SVD. Cependant, son coût de factorisation reste élevé : à chaque itération, la recherche du pivot global implique une complexité en $\mathcal{O}(kmn)$. Le coût de stockage, lui, reste identique à la SVD tronquée, soit $\mathcal{O}(k(m+n))$. Cette approche est donc trop coûteuse pour des matrices de grande taille.

2.3 Adaptive Cross Approximation, cas partiel (ACA_p)

La méthode ACA_p repose sur la même idée que l'ACA complète. Cependant, contrairement à l' ACA_c , la sélection du pivot n'est pas effectuée sur l'ensemble de la matrice, mais uniquement dans une ligne ou une colonne du résidu choisi à chaque itération.

Une implémentation naïve de cette méthode consiste à déterminer le pivot en le choisissant dans une ligne fixée de R . Il faut alors calculer R à chaque itération.

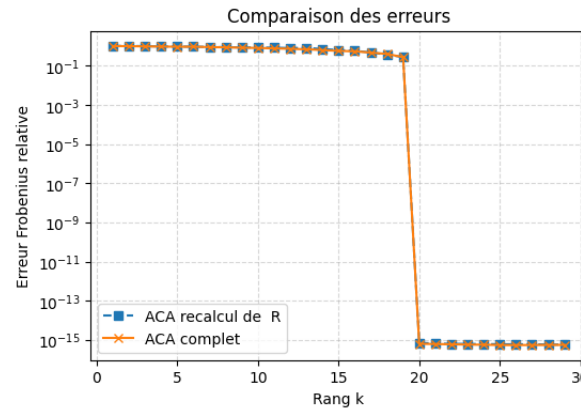


Figure 3 – Comparaison entre l'erreur en norme de Frobenius entre $A \in \mathbb{R}^{1000 \times 1000}$ de rang faible $r_f = 20$ pour $k \in [1, 2, \dots, 30]$ et son approximation A_k obtenue par ACA_c et $\text{ACA}_{naïf}$.

Nous avons donc commencé par cette approche, cette implémentation naïve donne une convergence similaire à celle de ACA_c (visible en Figure 3) mais donne des résultats décevants : le temps de calcul est presque aussi long que pour ACA_c (0,150sec pour ACA_c et 0,125sec pour cette méthode).

En pratique, recalculer le résidu complet à chaque étape est inutile, et on peut simplement déterminer le pivot à l'étape t à partir des vecteurs u_{t-1} et v_{t-1} construits à l'itération précédente.

Cette stratégie rend la méthode beaucoup plus rapide, avec un coût de factorisation en $\mathcal{O}(k(m+n))$. De plus le coût de stockage reste identique à celui de la SVD tronquée et de l'ACA complète, soit $\mathcal{O}(k(m+n))$.

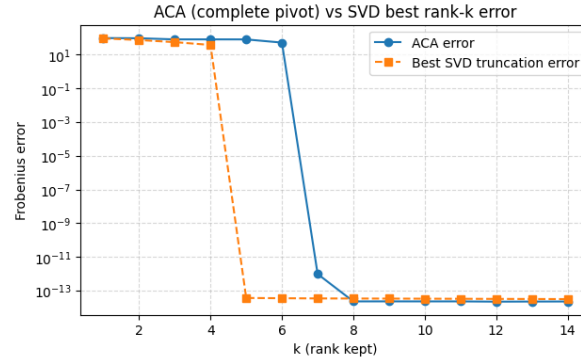


Figure 4 – Comparaison entre l'erreur en norme de Frobenius entre $A \in \mathbb{R}^{50 \times 50}$ de rang faible $r_f = 5$ pour $k \in [1, 2, \dots, 14]$ et son approximation A_k obtenue par ACA_p et SVD tronquée.

On observe que l'erreur décroît également mais plus tard que la SVD, autour du rang $k = 7$ (contre $k = 5$ pour la SVD). Cette convergence plus lente est largement compensée par la forte réduction du coût de calcul.

3 Comparaison des méthodes

3.1 Comparaison des méthodes

Nous souhaitons à présent comparer ces méthodes, en particulier le temps qu'elles mettent pour approximer une matrice de taille plus conséquente, ainsi que le cas où la matrice A , de rang faible, présente un faible bruit aléatoire sur l'ensemble de ses cases.

On rappelle les complexités associées à chaque méthode pour une matrice $A \in \mathbb{R}^{m \times n}$:

Méthode	Coût de factorisation	Coût de stockage
SVD tronquée	$\mathcal{O}(mn \min(m, n))$	$\mathcal{O}(k(m+n))$
ACA_p	$\mathcal{O}(k mn)$	$\mathcal{O}(k(m+n))$
ACA_c	$\mathcal{O}(k(m+n))$	$\mathcal{O}(k(m+n))$

Comparaison de l'erreur en norme de Frobenius entre $A \in \mathbb{R}^{1000 \times 1000}$ de rang faible $r_f = 20$ et A_k déterminée via SVD tronquée, ACA_p , ACA_c pour $k \in [1, 2, \dots, 30]$

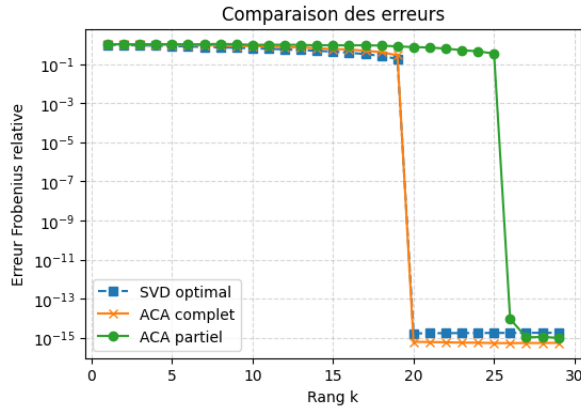


Figure 5 – cas où $\text{bruit} = 0$

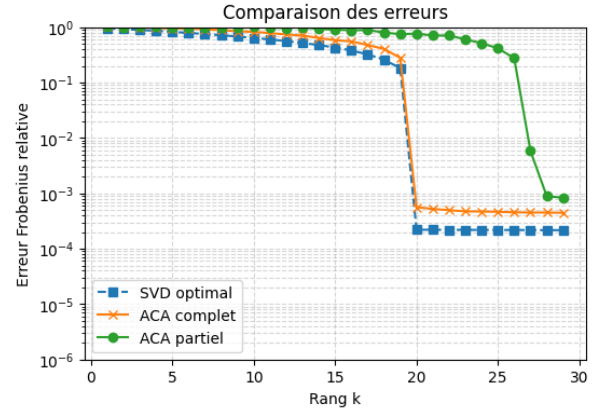


Figure 6 – cas où $\text{bruit} = 0,001$

On observe en Figure 5, comme précédemment, que l'erreur diminue substantiellement pour $k = 20$ pour la SVD et ACA_c et qu'il faut un peu plus, $k = 27$ pour que ACA_p voit son erreur diminuer suffisamment. Par ailleurs, en Figure 6, un bruit léger empêche d'ores et déjà la factorisation d'admettre une erreur aussi faible que précédemment (on considère une erreur au mieux en $\approx 10^{-6}$). Les temps de calcul obtenus pour un bruit égal à 0 sur la même matrice A que précédemment sont représentés dans le tableau ci-dessous. Le temps mesuré est arrêté à partir du moment où l'erreur est $< 10^{-6}$. Les valeurs affichées sont une moyenne effectuée sur 10 matrices A test.

Méthode	temps de calcul
SVD tronquée	0.252s
ACA_p	0.178s
ACA_c	0.005s

Les résultats obtenus sont cohérents avec les estimations théoriques : on a bien $t_{SVD} > t_{ACA_c} \gg t_{ACA_p}$. On observe que la méthode ACA_p est bien plus rapide, c'est donc celle là dont nous nous servirons pour le problème de Helmholtz.

3.2 Application à la matrice BEM

Dans le cadre de la méthode BEM, nous pourrions nous demander si l'une de ces méthodes serait satisfaisante pour approximer la matrice A du problème $Ax = b$. (*Remarque : pour les tests, la matrice BEM a été calculée avec 100 segments, pour une fréquence d'onde incidente $f_k = \pi$, pour une quadrature de gauss à 4 points sur l'extérieur d'un disque de rayon 1*)

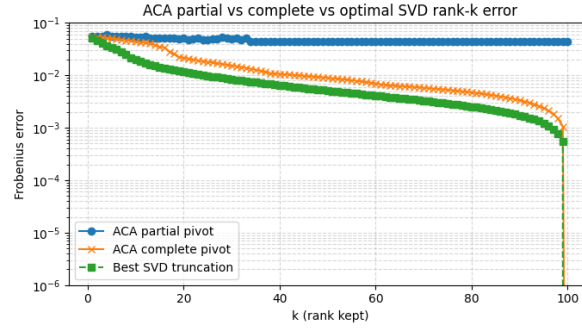


Figure 7 – Comparaison entre l’erreur en norme de Frobenius entre la matrice BEM $A \in \mathbb{R}^{100 \times 100}$ pour $k \in [1, 2, \dots, 100]$ et son approximation A_k obtenue par ACA_p et SVD tronquée.

Aucune méthode ne semble efficace, l’erreur reste très grande. Naturellement, si l’on prend $k=100$ l’erreur vaut 0, mais dans ce cas on ne réduit pas le problème étudié. Pour comprendre pourquoi ces méthodes ne fonctionnent pas, on décide de modéliser les valeurs singulières de A .

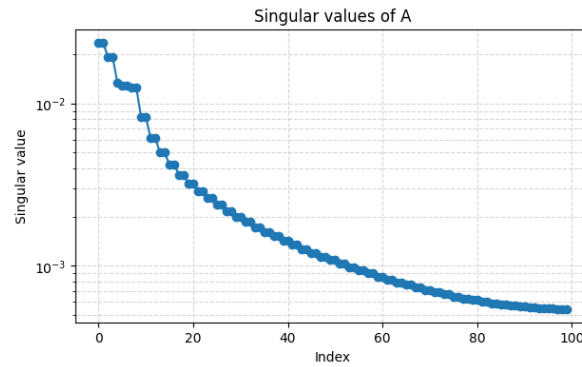


Figure 8 – valeurs singulières de la matrice BEM $A \in \mathbb{R}^{100 \times 100}$

Comme le montre la Figure 8, la matrice A n’est pas strictement de rang faible, ce qui empêche les méthodes d’approximations vues précédemment de converger efficacement. Il va donc sûrement nous falloir trouver une solution pour ne factoriser localement A que lorsqu’elle est de rang faible.

4 Conclusion

Ce travail a permis d’évaluer trois méthodes de factorisation de rang faible visant à accélérer la résolution du système linéaire dense issu de la BEM pour l’équation de Helmholtz. l’ACA avec pivotage partiel s’est révélée être la plus efficace, avec un coût de factorisation réduit à $\mathcal{O}(k(m+n))$ et un temps de calcul significativement inférieur aux autres méthodes étudiées, ce qui la positionne comme la méthode la plus prometteuse pour notre problème. Toutefois, l’application directe de ces méthodes à la matrice BEM montre une efficacité limitée. Cette matrice n’est pas globalement de rang faible, ce constat montre la nécessité d’une méthode supplémentaire pour implémenter cette factorisation à la matrice BEM.

4.1 Annexe : Utilisation de l'IA

L'Intelligence Artificielle nous a permis d'accélérer la construction des codes pour certaines des méthodes, et nous également servie à déboguer certains fragments de code. D'autre part, l'IA s'est révélée utile pour réexpliquer certains concepts théoriques de la méthode ACA_p mal compris précédemment.